

Министерство образования Республики Беларусь

Учреждение образования
«Белорусский государственный университет информатики и
радиоэлектроники»

Кафедра электронных вычислительных машин

Лабораторная работа №3
«Организация параллельной обработки запросов на сервере с помощью
мультиплексирования»

Выполнили:
студенты группы 250501
Шпаковский П.И.
Снитко Д.А.
Щербаков Д.И

Проверил:
Андриевский Е.С.

Минск 2025

СОДЕРЖАНИЕ

ЦЕЛЬ И ЗАДАЧИ	2
1 ДЕМОНСТРАЦИЯ РАБОТЫ	4
1.1 Запуск сервера и подключение клиентов	4
1.2 Параллельная обработка запросов клиентов	4
2 ОПИСАНИЕ ПРОГРАММНЫХ МОДУЛЕЙ	5
2.1 Сервер	5
2.2 Клиент	6
ЗАКЛЮЧЕНИЕ	7
ПРИЛОЖЕНИЕ А.....	8

ЦЕЛЬ И ЗАДАЧИ

Цель лабораторной работы – изучить методы параллельного обслуживания нескольких клиентов в рамках одного потока с использованием мультиплексирования.

Задачи лабораторной работы:

1. Модифицировать сервер для параллельного обслуживания клиентов с использованием мультиплексирования.
2. Реализовать обработку нескольких клиентов по протоколу ТСР.
3. Провести тестирование сервера и продемонстрировать его работу.

1 ДЕМОНСТРАЦИЯ РАБОТЫ

1.1 Запуск сервера и подключение клиентов

Для запуска сервера необходимо выполнить команду:

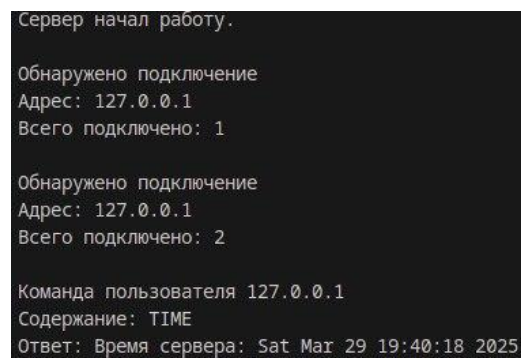
```
python3 server.py
```

Каждый клиент запускается отдельным процессом, который подключается к серверу.

Для запуска и подключения клиента необходимо выполнить команду:

```
python3 client.py
```

На рисунке 1.1 представлен результат запуска сервера, подключения двух клиентов и вывод команды TIME:

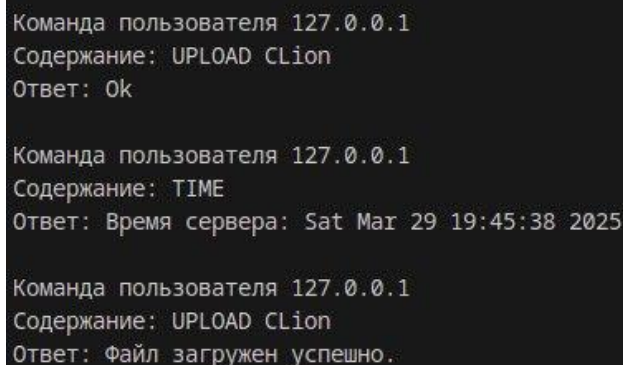


```
Сервер начал работу.  
Обнаружено подключение  
Адрес: 127.0.0.1  
Всего подключено: 1  
  
Обнаружено подключение  
Адрес: 127.0.0.1  
Всего подключено: 2  
  
Команда пользователя 127.0.0.1  
Содержание: TIME  
Ответ: Время сервера: Sat Mar 29 19:40:18 2025
```

Рисунок 1.1 – Запуск сервера и подключение клиентов

1.2 Параллельная обработка запросов клиентов

На рисунке 1.2 представлен результат “параллельного” выполнения команд DOWNLOAD и TIME.



```
Команда пользователя 127.0.0.1  
Содержание: UPLOAD CLion  
Ответ: Ok  
  
Команда пользователя 127.0.0.1  
Содержание: TIME  
Ответ: Время сервера: Sat Mar 29 19:45:38 2025  
  
Команда пользователя 127.0.0.1  
Содержание: UPLOAD CLion  
Ответ: Файл загружен успешно.
```

Рисунок 1.2 – Параллельное выполнение команд DOWNLOAD и UPLOAD

2 ОПИСАНИЕ ПРОГРАММНЫХ МОДУЛЕЙ

Код программы представлен в приложении А.

2.1 Сервер

Сервер использует библиотеку `socket` для работы с TCP-соединениями и `threading` для организации многопоточности. Ключевые компоненты:

Функция `setOptions(sock)` – настройка параметров сокета, таких как `SO_KEEPALIVE` (проверка активности соединения), `SO_REUSEADDR` (повторное использование адреса) и параметры TCP `KeepAlive`.

Функция `handleCommand(conn, clientSocket)` – обработка команд, полученных от клиента. Поддерживаемые команды:

`echo <текст>` – возвращает клиенту введенный текст.
`time` – отправляет клиенту текущее серверное время.
`download <файл>` – передача файла клиенту.
`upload <файл>` – загрузка файла на сервер.
`exit` – закрытие соединения с клиентом.

Функция `regClient(conn, addr)` – добавляет клиент в список сокетов для обработки:

Функция `main()` – главный цикл работы сервера, позволяет следить за несколькими сокетами одновременно и определять какие из них готовы для чтения или записи.

Для обеспечения одновременной работы с несколькими клиентами сервер использует механизм мультиплексирования ввода-вывода с помощью `select`. Основной серверный процесс принимает входящие соединения и отслеживает активность всех клиентов в одном потоке, обрабатывая чтение и запись данных по мере их готовности. Таким образом, несколько клиентов могут работать с сервером одновременно, без блокировки друг друга и без необходимости создавать отдельные потоки для каждого соединения. Когда клиент подключается:

1. Сервер вызывает `accept()`, создавая новый `clientSocket`.
2. Соединение добавляется в `connections`, чтобы `select` мог отслеживать его.
3. Серверный цикл обрабатывает клиентов в одном потоке, проверяя чтение, запись, ошибки.
4. Обработчик `handleCommand()` анализирует команды и выполняет их.
5. Передача файлов (`upload/download`) выполняется порциями, чтобы не блокировать других клиентов.
6. Если клиент отключается, соединение удаляется из списка `connections`.

Этот метод обеспечивает одновременное обслуживание нескольких клиентов без ожидания завершения работы одного перед началом работы другого.

2.2 Клиент

Клиентская программа реализует подключение к серверу и отправку команд. Ключевые компоненты:

Функция `connect(clientSocket)` – устанавливает соединение с сервером, применяя TCP KeepAlive для устойчивости соединения.

Функция `upload(filePath)` – отправляет файл на сервер по частям, используя контроль передачи с учетом размера файла и подтверждения получения сервером.

Функция `download(filePath)` – получает файл от сервера, продолжая загрузку с прерванного места в случае разрыва соединения.

Основной цикл ввода команд.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы был изучен и реализован сервер на основе ТСР-соединений с использованием мультиплексирования. Сервер успешно обрабатывает одновременные подключения клиентов в одном потоке, корректно выполняя команды загрузки и скачивания файлов. Применение механизма мультиплексирования позволило эффективно управлять сетевыми соединениями без создания отдельных потоков, обеспечивая стабильную и быструю работу сервера при обслуживании нескольких клиентов.

ПРИЛОЖЕНИЕ А

(обязательное)

Код программы

Файл server.py

```
import socket
import time
import os
import select

BIND_ADDRESS = "172.20.10.5"
BIND_PORT = 54321
OPT_INTERVAL = 10
OPT_COUNT = 3
FRAME_SIZE = 8192

def setOptions(clientSocket):
    clientSocket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    clientSocket.setsockopt(socket.SOL_SOCKET, socket.SO_KEEPALIVE, 1)
    clientSocket.setsockopt(socket.SOL_TCP, socket.TCP_USER_TIMEOUT, 30000)
    clientSocket.setsockopt(socket.IPPROTO_TCP, socket.TCP_KEEPIFIDLE,
OPT_INTERVAL)
    clientSocket.setsockopt(socket.IPPROTO_TCP, socket.TCP_KEEPIFINTVL,
OPT_INTERVAL)
    clientSocket.setsockopt(socket.IPPROTO_TCP, socket.TCP_KEEPCNT,
OPT_COUNT)

    return clientSocket

def downloadStart(conn, fileName, commandText):
    if not os.path.exists(fileName):
        conn.send("0".encode())
        return (False, "Файл \"" + fileName + "\" не найден.")

    conn.send("1".encode())

    file = open(fileName, 'rb')
    fileSize = os.path.getsize(fileName)
    offset = int(conn.recv(FRAME_SIZE).decode())
    conn.sendall(str(fileSize).encode() + b"\n")
    file.seek(offset, 0)

    socketsToWrite.append(conn)
    setFileProperties(conn, True, file, fileSize, offset, commandText)
    printStartFileLoading(conn, fileName, False)
    return (True, None)

def downloadFile(conn):
    data = properties[conn.fileno()][2].read(FRAME_SIZE)

    if data:
        conn.send(data)
        return False
    else:
        return True

def downloadEnd(conn):
    properties[conn.fileno()][2].close()
```



```

socketsToWrite.remove(conn)
command = properties[sock.fileno()][5]
setFileProperties(conn, False, None, None, None, "")
return command, "Файл передан успешно."

def uploadStart(conn, fileName, commandText):
    clientHasFile = conn.recv(1).decode()
    if clientHasFile == "0":
        return (False, conn.recv(FRAME_SIZE).decode())

    mode = 'ab' if os.path.exists(fileName) else 'wb+'

    file = open(fileName, mode)
    fileSize = int(conn.recv(FRAME_SIZE).decode())
    conn.send(str(os.path.getsize(fileName)).encode())
    offset = os.path.getsize(fileName)
    file.seek(0, os.SEEK_END)

    setFileProperties(conn, True, file, fileSize, offset, commandText)
    printStartFileLoading(conn, fileName, True)
    return (True, None)

def uploadFile(conn):
    fileno = conn.fileno()
    if properties[fileno][3] > properties[fileno][4]:
        data = conn.recv(min(FRAME_SIZE, properties[fileno][3]))
        properties[fileno][3] -= len(data)
        properties[fileno][2].write(data)

    if properties[fileno][3] > properties[fileno][4]:
        return False
    else:
        return True

def uploadEnd(conn):
    properties[conn.fileno()][2].close()
    command = properties[sock.fileno()][5]
    setFileProperties(conn, False, None, None, None, "")
    return command, "Файл загружен успешно."

def echo(data):
    return data

def _time():
    currentTime = time.asctime(time.localtime())
    return "Время сервера: " + currentTime

def exit():
    return "Отключение от сервера..."

def handleCommand(conn, clientInput):
    command, argument = clientInput.partition(" ")[::2]

    match (command.lower()):
        case "echo":
            response = echo(argument)

```

```

        case "time":
            response = _time()

        case "download":
            response = downloadStart(conn, argument, clientInput)

        case "upload":
            response = uploadStart(conn, argument, clientInput)

        case "exit":
            response = exit()

        case _:
            response = "Такой команды не существует!"

    return command, response

def printLog(clientInput, response, addr):
    print("Команда пользователя", addr[0])
    print("Содержание:", clientInput)
    print("Ответ:", response, "\n")

def printStartFileLoading(conn, fileName, dir):
    if dir:
        print(f"\nНачат приём файла {fileName}\n"
              f"Отправитель: {properties[conn.fileno()][0][0]}\n")
    else:
        print(f"\nНачата передача файла {fileName}\n"
              f"Получатель: {properties[conn.fileno()][0][0]}\n")

def unregClient(conn):
    del properties[conn.fileno()]
    connections.remove(conn)
    conn.close()
    print("Всего подключено:", len(connections) - 1, '\n')

def regClient(conn, addr):
    connections.append(conn)
    properties[conn.fileno()] = [addr, False, None, None, None, ""]
    print("Всего подключено:", len(connections) - 1, '\n')

def setFileProperties(conn, loading, fd, bytesRemaining, offset, commandText):
    properties[conn.fileno()] = [properties[conn.fileno()][0], loading, fd,
    bytesRemaining, offset, commandText]

try:
    connections = []
    socketsToWrite = []
    properties = {}

    serverSocket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    serverSocket = setOptions(serverSocket)
    serverSocket.bind((BIND_ADDRESS, BIND_PORT))
    serverSocket.listen()

    serverSocket.setblocking(False)

```

```

connections.append(serverSocket)

print("Сервер начал работу.\n")

while (True):
    readReady, writeReady, socketsWithErrors = select.select(connections,
socketsToWrite, connections)

    for sock in readReady:
        if sock == serverSocket:
            clientConn, clientAddr = serverSocket.accept()
            print("Обнаружено подключение\n" + "Адрес:", clientAddr[0])
            regClient(clientConn, clientAddr)
            continue

        try:
            if properties[sock.fileno()][1] == True:
                end = uploadFile(sock)
                if (end):
                    command, response = uploadEnd(sock)
                    sock.send(response.encode())
                    printLog(command, response,
properties[sock.fileno()][0])
                    continue

                clientInput = sock.recv(FRAME_SIZE).decode()
                if not clientInput:
                    print('Обрыв соединения с', properties[sock.fileno()][0])
                    if sock in socketsToWrite:
                        socketsToWrite.remove(sock)
                    unregClient(sock)
                    continue

                else:
                    command, response = handleCommand(sock, clientInput)
                    if ((command == 'upload' or command == 'download') and
response[0] == True):
                        continue

                    sock.send((response[1] if type(response) is tuple else
response).encode())
                    printLog(clientInput, response[1] if type(response) is
tuple else response,
                        properties[sock.fileno()][0])

                    if clientInput == 'exit':
                        print('Клиент', properties[sock.fileno()][0],
'отключился.')
                        unregClient(sock)

        except socket.error:
            print('\nОбрыв соединения с', properties[sock.fileno()][0])
            if sock in socketsToWrite:
                socketsToWrite.remove(sock)
            unregClient(sock)

    for sock in writeReady:
        try:
            if properties[sock.fileno()][1] == True:
                end = downloadFile(sock)
                if (end):
                    command, response = downloadEnd(sock)
                    sock.send(response.encode())

```

```

                                printLog(command, response,
properties[sock.fileno()][0])

        except socket.error:
            print('\nОбрыв соединения с', properties[sock.fileno()][0])
            if sock in socketsToWrite:
                socketsToWrite.remove(sock)
            unregClient(sock)

    for sock in socketsWithErrors:
        print('\nОбрыв соединения с', properties[sock.fileno()][0])
        if sock in socketsToWrite:
            socketsToWrite.remove(sock)
        unregClient(sock)

except KeyboardInterrupt:
    print("\nЗавершение работы.\n")

```

Файл client.py

```

import socket
import os
import time
import select

SERVER_ADDRESS = "172.20.10.5"
SERVER_PORT = 54321
OPT_INTERVAL = 10
OPT_COUNT = 3
BUF_SIZE = 1024

exitFlag = False

def setOptions(clientSocket):
    clientSocket.setsockopt(socket.SOL_SOCKET, socket.SO_KEEPALIVE, 1)
    clientSocket.setsockopt(socket.IPPROTO_TCP, socket.TCP_KEEPIIDLE,
OPT_INTERVAL)
    clientSocket.setsockopt(socket.IPPROTO_TCP, socket.TCP_KEEPIIDVL,
OPT_INTERVAL)
    clientSocket.setsockopt(socket.IPPROTO_TCP, socket.TCP_KEEPCNT, OPT_COUNT)

    return clientSocket

def connect(clientSocket):
    clientSocket = setOptions(clientSocket)
    clientSocket.connect((SERVER_ADDRESS, SERVER_PORT))
    return clientSocket

def reconnectPrompt():
    print("Соединение разорвано.",
        "\nХотите попробовать восстановить соединение? [y/n]")

    answer = input()
    if len(answer) <= 0:
        return False
    if (answer[0] == "Y") or (answer[0] == "y"):
        return True

    return False

def upload(filePath):
    if not os.path.exists(filePath):
        clientSocket.send("0".encode())

```

```

        clientSocket.send(("Файл \"" + filePath + "\"" не найден.").encode())
        return clientSocket.recv(BUF_SIZE).decode()

    clientSocket.send("1".encode())

    offset, fileSize = uploadFile(filePath)

    return clientSocket.recv(BUF_SIZE).decode()

def uploadFile(filePath):
    with open(filePath, 'rb') as file:
        fileSize = os.path.getsize(filePath)
        clientSocket.send(str(os.path.getsize(filePath)).encode())
        offset = int(clientSocket.recv(BUF_SIZE).decode())
        file.seek(offset, 0)

        while offset < fileSize:
            data = file.read(BUF_SIZE)
            clientSocket.send(data)
            fileSize -= len(data)

    file.close()
    return (offset, os.path.getsize(filePath))

def download(filePath):
    serverHasFile = clientSocket.recv(1).decode()
    if serverHasFile == "0":
        return clientSocket.recv(BUF_SIZE).decode()

    offset, fileSize = downloadFile(filePath)

    return clientSocket.recv(BUF_SIZE).decode()

def downloadFile(fileName):
    mode = 'ab' if os.path.exists(fileName) else 'wb+'

    with open(fileName, mode) as file:
        offset = os.path.getsize(fileName)
        clientSocket.send(str(offset).encode())

        size_data = b""
        while b"\n" not in size_data:
            size_data += clientSocket.recv(1)

        fileSize = int(size_data.decode().strip())

        file.seek(0, os.SEEK_END)

        while fileSize > offset:
            data = clientSocket.recv(BUF_SIZE if BUF_SIZE < fileSize - offset
            else fileSize - offset)
            fileSize -= len(data)
            file.write(data)

    file.close()
    return (offset, os.path.getsize(fileName))

def exit():
    global exitFlag
    exitFlag = True

def otherCommand(userInput):
    return clientSocket.recv(BUF_SIZE).decode()

```

```

def handleCommand(userInput):
    command, argument = userInput.partition(" ")[::2]

    match (command.lower()):
        case "upload":
            response = upload(argument)

        case "download":
            response = download(argument)

        case _:
            response = otherCommand(userInput)

    if (userInput.lower() == "exit"):
        exit()

    return response

try:
    clientSocket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    clientSocket = connect(clientSocket)
    print("Соединение установлено.\n")

    while not exitFlag:
        try:
            userInput = input("> ")
            while userInput == "":
                userInput = input("> ")
            clientSocket.send(userInput.encode())
            response = handleCommand(userInput)
            print(response, "\n")

        except socket.error:
            clientSocket.close()
            try:
                result = reconnectPrompt()
                if (result):
                    clientSocket = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
                    clientSocket = connect(clientSocket)
                    print("Соединение восстановлено.\n")
                else:
                    exitFlag = True
            except socket.error:
                print("Не удалось восстановить подключение.")
                exitFlag = True

    except socket.error:
        print("Сервер недоступен.\n")
    except KeyboardInterrupt:
        print("\nЗавершение работы.\n")

finally:
    clientSocket.close()

```